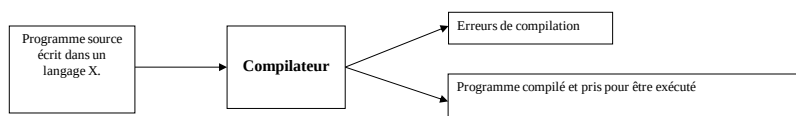


TP Compilation

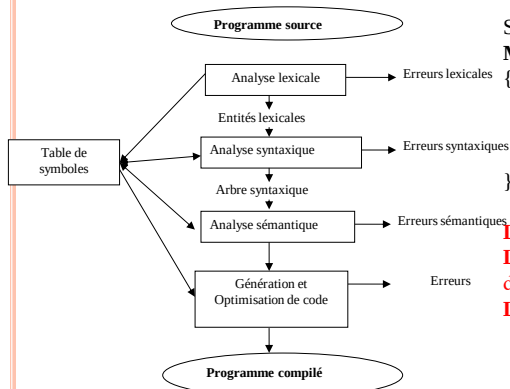
L'analyse lexicale

Structure d'un fichier FLEX

Introduction à la compilation



Plus en détail



Exemple

Soit le programme C suivant :

```

Main ()
{
  Int x, y ;
  If x==2) y=1;
  X=3,14;
}
  
```

Le pgm est correcte lexicalement,
Le pgm est syntaxiquement incorrecte : manque de la (dans l'instruction if
Le pgm sémantiquement incorrecte :
 x = 3,14

Analyse lexicale

- L'analyseur lexical constitue la première étape d'un compilateur.
- **Ses tâches principales sont :**
 - La lecture des caractères d'entrée et de la production des entités lexicales en sortie.
 - L'élimination de caractères superflus : les commentaires, les tabulations, fin de lignes, ...
 - Gérer les numéros de ligne dans le programme source afin de récupérer la ligne dans laquelle une erreur lexicale est apparue.



Analyse lexicale

- **Exemple :**
 - Soit le code source suivant avant compilation :


```
If (x==2)      y=z ; /* affectation*/
```
 - Après l'analyse lexicale

MotCle_IF (IDF Egal Const) IDF Aff IDF pvg



L'analyse lexicale

- L'analyseur lexical est basé sur l'algorithme simple suivant :

```

Lire (ChaineEntrée);
Switch (ChaineEntrée)
Case (ExpReg1) : coder (« Entité1 »);
Case (ExpReg2) : coder (« Entité2 »);
....
Case (ExpRegN) : coder (« EntitéN »);
Default : écrire (« Erreur lexicale »);

```

- L'implémentation de cet algorithme nécessitera l'écriture de centaines/milliers de lignes de code.

L'analyse lexicale « FLEX »

- **FLEX** est un générateur d'analyseur lexicale.
- Il traduit notre analyseur lexical écrit dans un langage simple (FLEX) vers la langage C



- **But :**
 - Reconnaître les entités lexicales du langage
 - Exécuter des actions a chaque entité rencontré

Structure d'un fichier FLEX

- Un document FLEX comporte trois parties importantes séparées par le symbole %%

%{ Définitions en langage C %}	}	Partie 1
Les définitions des expressions régulières		
%% Les règles de traduction { Action C}	}	Partie 2
%% Code additionnel : Redéfinitions des fonctions prédéfinies	}	Partie 3

Structure d'un fichier FLEX

- **La partie 1 : Les expressions régulières**
- **But :** Donner les ER des différentes entités lexicales sous la forme suivante :
 - `<identificateur_de_l'entité> <expression_régulière>`
- **identificateur_de_l'entité :** doit commencer par une lettre
- Il ne comporte que des caractères alphanumériques et des _ ou bien des -
- **expression_régulière** doit être valide

Structure d'un fichier FLEX

- La partie 1 : Les expressions régulières

caractères	significations	exemples
+	Répéter 1 ou plusieurs fois	x+ x...x
*	Répéter 0 ou plusieurs fois	t* vide ou t...t
?	0 ou 1 fois	a? vide ou a
	union	ab bc ab ou bc
()	factorisation	(a b)c ac ou bc
"	valeur littérale des caractères	"+"? +?...+?
\	valeur littérale de caractère qui le précède même entre ""	\++ +...+ "\"+ erreur car la première " ne se ferme pas.
.	N'importe quel caractère sauf la fin de ligne (\n).	.\n n'importe quel caractère

Structure d'un fichier FLEX

- La partie 1 : Les expressions régulières

caractères	significations	exemples
[...]	ensemble de caractères.	[aeiou] la voyale :a ou e ou i ou o ou u.
-	Utilisé dans [] signifie un intervalle d'ensemble	[0-9] 0 1 2 3 4 5 6 7 8 9
^	Utilisé dans [] signifie le complément d'ensemble	[^0-9] tout caractère sauf chiffre

Attention :

[...] = ensemble de caractères, pas d'expressions régulières, ce qui veut dire:
 [(0|1)+] un des caractères (,), 0, 1, |, +, pas une suite de 0 ou 1.

Mais les caractères ^, \, - restent des caractères spéciaux.

[^\]] tout caractère sauf]

Structure d'un fichier FLEX

- **Donner une expression régulière qui définit les entités suivantes :**

- Une suite de caractères alphanumériques qui commence par un caractère alphabétique.
- `[a-zA-Z]([a-zA-Z]|[0-9])*`
- Les constantes numérique signées par le (-/+).
- `[+-]?([1-9]|[0-9])*[0]`
- N'importe quel caractère (la fin de ligne (\n) est incluse).
- `.\n`
- Tous les caractères à part les espaces, les tabulations et les fins de lignes.

`[^ \n\t]`

Structure d'un fichier FLEX

- **Partie 2 : Les règles de traduction**

- **But :** présenter l'ensemble des actions associées à chaque entité lexicale sous la forme suivante :

- `<pattern> <action>`
- **Un pattern :** une expression régulière décrivant une entité lexicale
- **Une action :** c'est un code C qui sera exécuté à chaque fois qu'une entité lexicale apparaît
- **Exemple :**
- `{ entier } { printf (" L'entité reconnu est un entier" ) ; }`

Structure d'un fichier FLEX

- **Exemple 1:** Un analyseur lexical pour le langage :

- `X=5 ;`

```
%{
    int nb_ligne=1;
}%
lettre [a-zA-Z]
chiffre [0-9]
IDF {lettre}({lettre}|{chiffre})*
cst {chiffre}+
%%
{IDF} printf ("IDF \n");
{cst} printf ("cst");
"=" printf ("aff");
";" printf ("pvg");
[ \t]
\n {nb_ligne++; }
. printf("erreur lexicale à la ligne %d \n",nb_ligne) ;
%%
Int main ()
{
    yylex () ;
    return 0 ;
}
```

Structure d'un fichier FLEX

- **Exemple 2 :** Un analyseur lexical pour le langage :

- `X=5 ;`
- `Y=4.5`

```
%{
    int nb_ligne=1;
}%
lettre [a-zA-Z]
chiffre [0-9]
IDF {lettre}({lettre}|{chiffre})*
cst {chiffre}+
Réal {chiffre}+ "." {chiffre}+
%%
{IDF} printf ("%s", yytext) ;
{réel} { printf ("%s", yytext) ; }
{cst} printf ("%s", yytext) ;
= printf ("aff") ;
; printf ("pvg");
[ \t]
\n {nb_ligne++; }
. printf("erreur lexicale à la ligne %d \n", nb_ligne) ;
%%
Int main ()
{
    Yylex () ;
    return 0 ;
}
```


Commandes de compilation

- Pour compiler le programme :
 - **flex MonTP.l**
 - **cc lex.yy.c -o MonTP -lfl (Linux)**
 - **gcc lex.yy.c -o MonTP -lfl (Windows)**
- Pour exécuter le programme :
 - **./MonTP**
- Pour arrêter le programme :
 - **Ctrl + c**

Macro-action de FLEX

Fonctions	yylex	Permet de lancer l'analyseur lexical
	yywrap	Elle est appelée par le Lexer quand il rencontre la fin du fichier. Elle doit soit obtenir un nouveau flux d'entrée et retourner simplement la valeur 0 soit renvoyer 1 signifiant que la totalité des flux a été consommée et que le lexer a fini sa tâche.
	yyterminate	Permet de provoquer la fin d'exécution du lexer
	ECHO	Affiche l'unité lexicale reconnue
Variables	yytext	Récupère le texte formant le lexème reconnu
	yylen	Détermine la longueur du texte contenue dans yytext
	yyval	Est une variable globale utilisé par FLEX pour stocker la valeur correspondante au Token reconnu
	yylineno	Est le numéro de la ligne courante
	yyin	Fichier d'entrée
	yyout	Fichier de sortie

Exemple

```
%{
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include "TS.h"
int nb_ligne=1;

%}

lettre [a-zA-Z]
chiffre [0-9]
IDF {lettre}({lettre}|{chiffre})*
cst {chiffre}+
%%
{IDF}          printf(" IDF reconnu %s
\n",yytext);
{cst}          printf(" CST reconnu\n");
"="           printf(" = reconnu\n");
";"           printf(" ; reconnu\n");
[ \t]
\n nb_ligne++;
. printf("erreur lexicale à la ligne %d \n",nb_ligne);
%%

int main()
{
yyin = fopen( "programme.txt", "r" );
if (yyin==NULL) printf("ERROR \n");
else yylex();
}
```

L'analyse syntaxique

Analyse syntaxique

L'analyse syntaxique vérifie que les unités lexicales (le résultat de l'analyse lexicale) sont dans le bon ordre défini par le langage.

Exemple : Dans le langage C

If X==2) y=1;

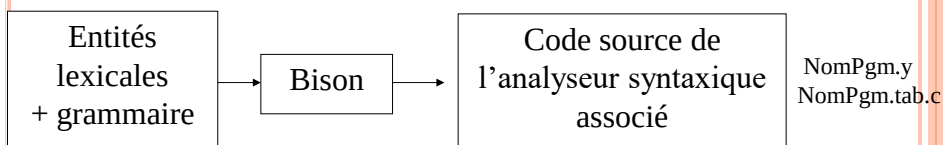
est une erreur de syntaxe car la règle de if impose une parenthèse avant la condition

Implémenter un analyseur syntaxique nécessite l'implémentation d'une méthode d'analyse syntaxique vue en cours LL(k), LR(k), SLR(k), LALR(k), ..etc. Ceci nécessite l'écriture de milliers de lignes de code.

Heureusement un outil tel que Bison existe pour nous faciliter la tâche.

Bison

- Bison est un générateur de code d'analyseur syntaxique.
- Il accepte comme entrée la sortie de l'analyse lexicale (les entités lexicales), et la grammaire de langage à analyser (les fichiers bison portent l'extension « .y »).



Format d'un fichier Bison

Le format d'un fichier Bison est similaire à celui de Flex. Il est composé de trois parties séparées par '%%'.

Structure d'un fichier Bison

%{

Définitions en langage C

%}

Les définitions des terminaux et d'axiome

%%

Les règles de la grammaire

%%

Redéfinitions des fonctions prédéfinies

Partie 1

Partie 2

Partie 3

Structure d'un fichier BISON

o La partie 1:

2.1. Déclaration C (pré-code)

Cette partie peut contenir les en-têtes, les macros et les autres déclarations C nécessaires pour le code de l'analyseur syntaxique.

2.2. Définitions et options

Cette partie contient toutes les déclarations nécessaires pour la grammaire à savoir:

- a) **Déclaration des symboles terminaux**
- b) **Définition des priorités et d'associativité**
- c) **Autres déclarations :**

A. Déclaration des symboles terminaux

ceci est effectué en utilisant le mot clé %token.

Exemple :

```
%token MAIN IDF Accolade PointVirgule
```

On peut préciser le type d'un terminal par

```
%token<type>.
```

Exemple :

```
%token<int> entier
```

```
%token<chaine> entier
```

B. Définition des priorités et d'associativité

l'associativité est définie par les mots clé :

```
%left,
```

```
%right et
```

```
%nonassoc.
```

la priorité est définie selon l'ordre de déclaration des unités lexicales.

Exemple :

```
%left A B /*associativité de gauche à droite*/ -
```

```
%right C D /* associativité de droite à gauche*/
```

Ordre de priorité

```
%nonassoc E F /* pas d'associativité*/ +
```

C. Autres déclarations

%start : permet de définir l'axiome de la grammaire.

- En l'absence de cette déclaration, Bison considère le premier non-terminal de la grammaire en tant que son axiome.
- **%type** : définir un type à un symbole non-terminal.
- **%union** : permet de spécifier tous les types possibles pour les valeurs sémantiques.

Structure d'un fichier BISON

o La partie 2: les règles de production

Ici, on décrit la grammaire LALR(1) du langage à compiler et les routines sémantiques à effectuées selon la syntaxe suivante :

<symbole NonTerminal> : <règle de dérivation 1>
 {action 1 en langage C }

- | <règle de dérivation 2> {action 2 en langage C }
- | ...
- | <règle de dérivation N> {action N en langage C }

;

Structure d'un fichier BISON

○ **Partie 3:** Post-code C

C'est le code principal de l'analyseur syntaxique. Il contient le main ainsi que les définitions des fonctions.

- elle doit contenir au minimum les deux fonctions suivantes.

```
main ()
{ yyparse(); }
yywrap ()
{
```

Lien entre FLEX et BISON

- Afin de lier FLEX à BISON, On doit ajouter dans la partie C du FLEX l'instruction suivante:

```
%{
#include "NomPgm.tab.h"
%}
```

Exemple

Création de compilateur lexical/syntaxique pour le langage : `x=5;`

Lien entre FLEX et BISON

Exemple

Création de compilateur lexical/syntaxique pour le langage:
x=5;

```

%{
#include "Syntaxique.tab.h"
int nb_ligne=1;
%}
lettre [a-zA-Z]
chiffre [0-9]
IDF {lettre}({lettre} | {chiffre})*
cst {chiffre}+
%%
{IDF} return idf;
{cst} return cst;
= return aff;
; return pvg;
[ \t]
\n {nb_ligne++;}
. printf("erreur lexicale à la ligne %d \n",nb_ligne) ;

```

Lexical.l

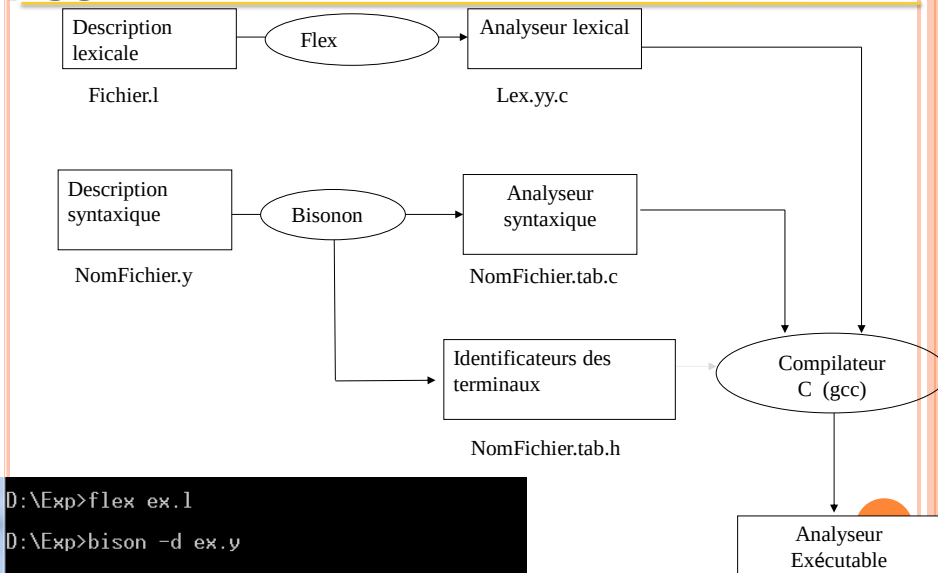
```

%token idf cst aff pvg
%%
S: idf aff cst pvg {printf("syntaxe correcte");
YYACCEPT;}
;
%%
int yyerror(char *msg)
{ printf("%s error syntaxique");
return 1; }
main ()
{
yyparse();
}
yywrap()
{}

```

Syntaxique.y

Commande de compilation et d'exécution FLEX/ BISON



```

D:\Exp>flex ex.l
D:\Exp>bison -d ex.y
D:\Exp>gcc lex.yy.c ex.tab.c -lfl -ly

```


Table des symboles

- La table de symboles doit être programmée manuellement dans flex et bison.
- La table de symboles doit contenir des informations sur les entités

```
PROGRAM TpComp
integer x;
Const reel y=5;
BEGIN
x=x+1;
END;
```

➔

state	name	code/nature	Type	val	Taille
1	TpComp	IDF			
1	X	IDF			
1	Y	IDF			
1	5	const	int	5	
	1	const	reel	1	

La table des symboles

- Chaque entité reconnue par le langage doit être insérée dans la table des symboles.

```
typedef struct
{
    int state;
    char name[20];
    char code [20];
    char type[20];
    float val;
} element;
```

element tab[1000];

La table des symboles des
constantes, variables

```
typedef struct
{
    int state;
    char name[20];
    char type[20];
} elt;
```

- **Name** : Nom de l'entité
- **Code** : variable simple, tableau, constante ...etc
- **Type** : type de l'entité (INT, Float ...etc)

elt tabs[40], tabm[40];

La table des symboles des
séparateurs et des mots clés

Fonction d'initialisation de la TS

```
void initialisation()
{
    int i;
    for (i=0;i<1000; i++)  Tab[i]. state=0;
    for (i=0;i<50;i++)
    {
        TabS[i].state=0;
        TabM[i].state=0;
    }
}
```

La fonction de recherche dans la TS

- Permet de vérifier si l'entité existe déjà dans la TS

```
void rechercher (char entite[ ], char type [ ], float val, int y)
```

```
{
    int j,i;
    Switch (y)
    {
        Case 0: /*verifier si la case dans la tables des IDF et CONST est libre*/
        FOR (i=0;(
            ( i<1000)      &&
            (Tab[i].state==1)) &&
            (strcmp(entite, Tab[i].name)!=0)
            ; i++);
        IF (i<1000)
            Insérer ( entite, type, val, i , 0);
        ELSE
            printf("entité existe déjà\n");
        break;
    }
```

Position d'insertion

0 Recherche dans TS des idf et des constantes
1 dans la TS des séparateurs
2 dans la TS des mots clés

La fonction de recherche dans la TS

Case 1: /*verifier si la case dans la tables des mots clés est libre*/

```
FOR (i=0;((i<40)&&(TabM[i].state==1))&&(strcmp(entite,Tab[i].name)!=0);i++);
```

```
IF (i<40)
```

```
    inserer(entite,type,val,i,1);
```

```
ELSE
```

```
    Printf ("entité existe déjà\n");
```

```
break;
```

La fonction de recherche dans la TS

Case 2: /*verifier si la case dans la tables des séparateurs est libre*/

```
FOR
```

```
(i=0;((i<40)&&(TabS[i].state==1))&&(strcmp(entite,TabS[i].name)!=0);i++);
```

```
IF(i<40)
```

```
    inserer(entite,type,val,i,2);
```

```
ELSE
```

```
    printf("entité existe déjà\n");
```

```
break;
```

```
} /* fin switch
```

```
} /* fin recherche
```

La fonction d'insertion dans la TS

```

void inserer (char entite[], char type[], float val, int i, int y)
{
    switch (y)
    {
        Case 0: /*insertion dans la table des IDF et CONST*/
        Tab[i].state=1;
        Strcpy (Tab[i].name, entite);
        Strcpy (Tab[i].type, type);
        tab[i].val=val;
        break;

        Case 1: /*insertion dans la table des mots clés*/
        TabM[i].state=1;
        Strcpy (TabM[i].name,entite);
        Strcpy (TabM[i].type,type);
        break;

        Case 2: /*insertion dans la table des séparateurs*/
        TabS[i].state=1;
        Strcpy (TabS[i].name,entite);
        Strcpy (TabS[i].type,type);
        break;
    }
}

```

TS.h

```

void initialisation();
{
}

#include <stdio.h>
#include <stdlib.h>

typedef struct
{
    int state;
    char name[20];
    char type[20];
    float val;
    int sub;
} element;

void inserer (char entite[], char type[],float val,int i,int y)
{
}

void recherche (char entite[], char type[],float val,int y)
{
}

typedef struct
{
    int state;
    char name[20];
    char type[20];
} elt;

void afficher()
{
}

element tab[1000];
elt tabs[40],tabm[40];

```

Exemple

```
%{
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include "TS.h"
int yylineo=1;
int Col=1;
%}
lettre [a-zA-Z]
chiffre [0-9]
idf
{lettre}({lettre}{chiffre})*
entier {chiffre}+
real {chiffre}+"."{chiffre}+
blanc [ \t]
SI [ \n]
%%
```

```
"/" {
    rechercher (yytext,"sep",0,2);
    Col = Col + strlen(yytext);
}

"=" {
    rechercher (yytext,"sep",0,2);
    Col = Col + strlen(yytext);
}

{idf} {
    Rechercher (yytext," ",0,0);
    Col = Col + strlen(yytext);
}

{real} {
    rechercher (yytext,"real",atof(yytext),0);
    Col = Col + strlen(yytext);
}

{entier} {
    rechercher (yytext,"entier",atoi(yytext),0);
    Col = Col + strlen(yytext);
}

{blanc} {Col = Col + strlen(yytext);}

{SI} {yylineo++;Col=1;}
```

Exemple(suite de la partie 2)

```
. { printf ("\n Erreur lexical: Ligne: %d; Collone: %d; Entité << %s >> non reconnu par le langage
\n",yylineo, Col, yytext);
Col = Col + strlen(yytext);
}

%%
int main()
{
    initialisation();
    yyin = fopen( "input.txt", "r" );
    if (yyin==NULL) printf("ERROR \n");
    else yylex();
    afficher();
    return 0;
}
```

Variables et fonctions prédéfinies de FLEX

- **char yytext[]**: tableau de caractères qui contient la chaîne d'entrée en cours d'analyse.

```
IDF [a-zA-Z]([a-zA-Z]|[0-9])*
%%
{IDF} {printf ("idf : "); printf ("%s ",yytext) ;}
```

```
var
x
```

PgmSource

```
Idf : var
Idf : x
```

PgmCompilé

- **int yylen**: retourne la longueur de la chaîne d'entrée en cours d'analyse.

```
IDF [a-zA-Z]([a-zA-Z]|[0-9])*
%%
{IDF} {if (yylen <=7) printf (" idf ");
      else printf ("erreur lexcale : idf trop long");
}
```

```
var
vartoplogue
```

PgmSource

```
Idf
Erreur lexcale :idf trop long
```

PgmCompilé

Variables et fonctions prédéfinies de FLEX

- **yylex()**: c'est la fonction qui lance l'analyseur lexical. Si on change le main on doit pas oublier de l'ajouter dans le main

```
%{
int nb_ligne=0;
}%
%%
\n nb_ligne++;
%%
int main()
{
yylex();
printf("nombre de ligne
%d",nb_ligne);
}
```

```
Var
X
Y
z
```

PgmSource

```
Nombre de
ligne 4
```

PgmCompilé

Variables et fonctions prédéfinies de BISON

YYACCEPT : instruction qui permet de stopper l'analyseur syntaxique en cas de succès.

main () : elle doit appeler la fonction `yyparse ()`. L'utilisateur doit écrire son propre `main` dans la partie du bloc principal.

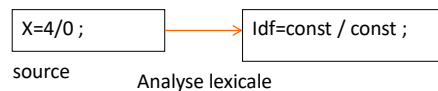
yyparse () : c'est la fonction principale de l'analyseur syntaxique. On doit faire appel à cette fonction dans la fonction `main()`.

int yyerror (char* msg) : lorsque une erreur syntaxique est rencontrée, `yyparse` fait appel à cette fonction. On peut la redéfinir pour donner plus de détails dans le message d'erreur. Par défaut elle est définie comme suit:

```
int yyerror ( char* msg ) {
    printf ( " Erreur Syntaxique rencontrée\n " );
}
```

Partage de variable entre FLEX et BISON

Comment Bison peut traiter les valeurs des entités lexicales, sachant que l'analyseur lexical à coder tout les entités de programme source?



Solution

Flex peut envoyer la valeur d'une entité avant de la coder en utilisant la variable prédéfinie **yyval**

Utilisation de yylval

1) Déclaration

Déclaration de **yylval** dans **FLEX**

```
%{
extern YYSTYPE yylval;
%}
```

Déclaration de type **YYSTYPE** dans **BISON**
Comme type composé

```
%union {
int    entier;
char*  str;
}
```

2) Utilisation

Envoie de valeur par FLEX

```
{cst} {
    yylval.entier=atoi(yytext);
    return cst;
}

{idf}
{yylval.str=strdup(yytext);
    return idf;
}
```

Traitement des valeurs par BISON

```
%token <str>idf  < entier>cst
%%;
s : idf= idf '/' cst if ($5==0) printf(" erreur : division par 0")
    ELSE printf("la Division est%s= %s/%d", $1,$3,$5);
    YYACCEPT;}
```

-Le symbole \$\$ référence la valeur associée au non terminal de la partie gauche d'une règle de grammaire.

-Le symbole \$i référence la valeur associée au i^{ème} symbole terminal ou non terminal de la partie droite d'une règle de grammaire.